

Stock Market Analytics Project

Complete Interview Questions, Answers & Follow-Ups

Based on your actual architecture and etl.py: Alpha Vantage → Python/Pandas → MySQL → Power BI → Windows Task Scheduler.

Q1. Tell me about your project.

Answer: I built an end-to-end stock market analytics pipeline. Alpha Vantage supplies daily data for RELIANCE.BSE, TCS.BSE, INFY.BSE and IBM. Python extracts, validates and transforms the data, MySQL stores it, Power BI visualizes it, and Windows Task Scheduler automates the ETL.

Follow-up questions + short answers:

- **Why this project?** — It demonstrates API integration, Python ETL, SQL, database design, BI and automation in one real-world workflow.
- **What was difficult?** — API rate limits, database connectivity, duplicate prevention and making the ETL reliable were key challenges.

Q2. Explain the architecture.

Answer: The flow is Alpha Vantage API → Python ETL → MySQL → Power BI, with Task Scheduler triggering the Python ETL. Python is responsible for extraction and transformation; MySQL provides persistent storage and analytics; Power BI provides reporting.

Follow-up questions + short answers:

- **Why MySQL in the middle?** — It provides persistent, queryable and reusable data instead of making the dashboard depend directly on an external API.
- **How would you scale it?** — Use incremental loading, bulk inserts, stronger validation, retries/backoff, indexes and orchestration.

Q3. What are the ETL stages?

Answer: Extract gets data from Alpha Vantage. Transform converts JSON to a DataFrame, converts types and removes invalid required rows. Load inserts cleaned records into MySQL.

Follow-up questions + short answers:

- **Which function extracts?** — `extract_stock_data()`.
- **Which function loads?** — `insert_new_records()`.

Q4. Why Python?

Answer: Python has mature libraries for HTTP requests, data processing, databases, testing and automation. Requests, Pandas, mysql.connector and python-dotenv fit the pipeline naturally.

Follow-up questions + short answers:

- **Why not SQL only?** — SQL cannot by itself conveniently handle the external API extraction and Python-side orchestration used here.

Q5. What does API_URL mean?

Answer: API_URL stores the API endpoint—the address where the HTTP request is sent. The params dictionary describes what data the program wants.

Follow-up questions + short answers:

- **What is an endpoint?** — A specific API URL through which a client communicates with a service.
- **What are parameters?** — Key-value inputs such as function, symbol, outputsize and API key.

Q6. Why use load_dotenv()??

Answer: It loads configuration from .env into environment variables, keeping secrets and environment-specific settings outside the source code.

Follow-up questions + short answers:

- **Should .env be committed?** — No. It should normally be excluded with .gitignore.
- **How manage secrets in production?** — Use a secrets manager or secure environment configuration.

Q7. Why use os.getenv()?

Answer: It retrieves values from environment variables. The code supplies defaults for non-secret settings such as localhost and root.

Follow-up questions + short answers:

- **What if a variable is missing?** — getenv returns None unless a default is supplied.

Q8. What does if __name__ == '__main__' mean?

Answer: It calls main() only when the file is executed directly, not when the module is imported.

Follow-up questions + short answers:

- **Why useful?** — It makes the file reusable without automatically starting the ETL on import.

Q9. How does the Alpha Vantage request work?

Answer: The script builds params and calls requests.get(API_URL, params=params, timeout=30). It then checks the HTTP response and parses JSON.

Follow-up questions + short answers:

- **Why GET?** — GET is appropriate for retrieving data without modifying the server state.
- **Why timeout?** — It prevents the program from waiting indefinitely.

Q10. What does raise_for_status() do?

Answer: It raises a Requests exception for unsuccessful HTTP status codes.

Follow-up questions + short answers:

- **Does it catch rate limits?** — Not necessarily. The provider may return an HTTP-success response containing a rate-limit message, so the script separately checks the JSON.

Q11. Why response.json()?

Answer: It parses the JSON response into Python dictionaries/lists so the program can access the time-series fields.

Follow-up questions + short answers:

- **What if JSON is invalid?** — The ValueError handler logs the problem and returns None.

Q12. Why check Time Series (Daily)?

Answer: Because an HTTP request can succeed while the API response contains a rate-limit note, information message or error message instead of stock data.

Follow-up questions + short answers:

- **How detect a rate limit?** — The code checks for the Note key and logs its message.

Q13. Why outputsize='compact'?

Answer: It requests a smaller response, which is sufficient for the current project and reduces transferred data.

Follow-up questions + short answers:

- **When use full?** — When a larger historical range is required and the API plan supports it.

Q14. Why wait 15 seconds?

Answer: It separates API requests to reduce the likelihood of rate-limit problems. The value is configurable and should match the provider's current limits.

Follow-up questions + short answers:

- **Better approach?** — Use retries with exponential backoff and provider-aware rate-limit handling.

Q15. Why convert JSON to a DataFrame?

Answer: A DataFrame gives a structured table for cleaning, type conversion, validation and loading.

Follow-up questions + short answers:

- **Why not insert JSON directly?** — A DataFrame makes column-level transformations and validation much easier.

Q16. What does `records.append({...})` do?

Answer: It creates one dictionary representing one stock-date row and adds it to a list. The list is then converted to a DataFrame.

Follow-up questions + short answers:

- **What does `time_series.items()` do?** — It yields each date key and its corresponding price-value dictionary.

Q17. Why convert dates with `pd.to_datetime().dt.date`?

Answer: The API provides dates as text. Parsing them gives a proper date representation for database storage and comparison.

Follow-up questions + short answers:

- **Why not keep strings?** — Proper date types support reliable sorting, filtering and database date operations.

Q18. Why `pd.to_numeric(errors='coerce')`?

Answer: The API values are strings. `pd.to_numeric` converts them to numbers, while `errors='coerce'` turns invalid values into NaN instead of crashing.

Follow-up questions + short answers:

- **What is NaN?** — A missing/not-a-number value used by Pandas for unavailable numeric data.

Q19. Why `dropna()`?

Answer: It removes rows missing essential fields: symbol, date or close.

Follow-up questions + short answers:

- **Why not drop every NaN?** — Some fields may be optional depending on the data-quality policy; required fields should be defined deliberately.

Q20. Why MySQL?

Answer: MySQL provides persistent relational storage, SQL analytics, transactions, constraints and a database connection that Power BI can use.

Follow-up questions + short answers:

- **Why not CSV?** — CSV is less suitable for concurrent querying, constraints, indexing and persistent analytical workflows.

Q21. Explain `stock_prices`.

Answer: It stores one stock observation per trading date, with symbol, date, open, high, low, close and volume.

Follow-up questions + short answers:

- **What key would you use?** — A unique constraint on symbol and date is appropriate for one daily record per stock.

Q22. Why use a cursor?

Answer: The cursor is the interface used to execute SQL statements through the MySQL connection and inspect execution results.

Follow-up questions + short answers:

- **Connection vs cursor?** — The connection represents the database session; the cursor executes statements within that session.

Q23. Explain WHERE NOT EXISTS.

Answer: The insert supplies the row values but only proceeds if a row with the same symbol and date does not already exist.

Follow-up questions + short answers:

- **Why not only Python duplicate checking?** — Database-side checking is closer to the data and can be backed by a UNIQUE constraint.

Q24. Why %s placeholders?

Answer: They are parameter placeholders used by the MySQL connector. Values are passed separately to `execute()`, avoiding unsafe SQL string concatenation.

Follow-up questions + short answers:

- **What is SQL injection?** — A security vulnerability where untrusted input changes the meaning of a SQL statement.

Q25. Why commit()?

Answer: It makes the successful database changes permanent for the current transaction.

Follow-up questions + short answers:

- **What is rollback?** — It reverses uncommitted changes in the current transaction.

Q26. What happens to duplicate rows?

Answer: WHERE NOT EXISTS evaluates false, so no row is inserted and rowcount does not increase.

Follow-up questions + short answers:

- **How strengthen this?** — Add a UNIQUE constraint/index on symbol and date.

Q27. Why logging instead of print()?

Answer: Logging provides levels such as INFO, WARNING and ERROR and is more useful for scheduled ETL monitoring and troubleshooting.

Follow-up questions + short answers:

- **Where store logs?** — In production, usually in rotating files or a centralized logging system.

Q28. Why try/except?

Answer: It lets the program handle expected failures such as network, JSON and database errors and log useful context.

Follow-up questions + short answers:

- **Why not catch Exception everywhere?** — Broad catches can hide programming bugs; catch specific expected exceptions where possible.

Q29. Why finally?

Answer: finally runs even when an unexpected exception occurs, so the database connection can be closed safely.

Follow-up questions + short answers:

- **Alternative?** — A context manager can make resource management cleaner in some cases.

Q30. What if one stock fails?

Answer: An expected extraction failure returns None, so that stock is skipped and the next stock can be processed.

Follow-up questions + short answers:

- **Should every error be skipped?** — No. Critical database or configuration failures should normally stop the run or trigger an alert.

Q31. Why Power BI?

Answer: Power BI provides interactive dashboards, slicers, KPIs, trends and business-friendly reporting.

Follow-up questions + short answers:

- **Why not Python charts?** — Power BI is stronger for interactive business reporting and distribution.

Q32. How does Power BI get data?

Answer: It connects to MySQL and reads the relevant tables or analytical data for the report.

Follow-up questions + short answers:

- **Import vs DirectQuery?** — Import loads data into the Power BI model; DirectQuery queries the source when visuals need data. The choice depends on freshness and scale.

Q33. Explain the three dashboard pages.

Answer: Market Overview summarizes the tracked market; Stock Performance focuses on individual stocks and comparisons; Risk & Trading focuses on volatility, returns and volume.

Follow-up questions + short answers:

- **Why separate pages?** — Each page answers a different analytical question and keeps the report easier to navigate.

Q34. How will you automate it?

Answer: Windows Task Scheduler launches the Python interpreter and etl.py on a schedule, removing the need to manually run the pipeline.

Follow-up questions + short answers:

- **What if the PC is off?** — The task cannot execute unless configured on an available machine/server; production systems should run on reliable always-on infrastructure.
- **How detect failure?** — Use logs plus scheduled-task status and, ideally, alerting.

Q35. What if you had 1,000 stocks?

Answer: The current sequential design would be too slow and could hit API limits. I would add incremental extraction, backoff/retries, batching, bulk database writes and orchestration.

Follow-up questions + short answers:

- **Would multithreading solve it?** — It can improve throughput only when the API limits and architecture allow it; concurrency cannot bypass a strict provider rate limit.

Q36. What if ETL runs twice?

Answer: The duplicate check prevents existing symbol/date records from being inserted again. A UNIQUE database constraint makes this safer.

Follow-up questions + short answers:

- **What is idempotency?** — Repeating the same operation produces the same final state rather than creating unintended duplicates.

Q37. What is a weakness of the current script?

Answer: Row-by-row iterrows() insertion and sequential API requests are simple but not optimal for large scale. Production improvements include bulk inserts, constraints, retries, tests, monitoring and better deployment.

Follow-up questions + short answers:

- **Which improvement first?** — A UNIQUE constraint and reliable error/retry policy are high-value foundations; bulk loading becomes important as volume grows.

Tricky Questions You Should Definitely Know

Q38. What if HTTP status is 200 but API returns an error?

Answer: HTTP success does not guarantee successful application data. The script checks for the expected time-series key and handles Note, Information and Error Message.

Q39. What if volume is 'abc'?

Answer: `to_numeric(..., errors='coerce')` produces NaN. Because volume is not currently in dropna's required subset, `int(volume)` could later fail. Stronger validation should check volume before insertion.

Q40. What if open/high/low are missing?

Answer: The current code can retain NaN. A production schema should decide whether these fields are mandatory and validate accordingly.

Q41. Why not SELECT first in Python?

Answer: It adds another round trip and can create a race condition. A database UNIQUE constraint is stronger.

Q42. What if two ETL jobs run at the same time?

Answer: Application-level NOT EXISTS checks can race. A UNIQUE constraint on symbol/date is the proper database-level protection.

Q43. Why is iterrows() not ideal for large data?

Answer: It loops row by row in Python and is slower than batch-oriented operations. `executemany` or bulk loading is preferable.

Q44. What if MySQL fails halfway?

Answer: The current function commits after the loop. For stronger production behavior, define a rollback/retry policy and distinguish recoverable row errors from fatal connection errors.

Rapid-Fire Concepts — With Answers

API

A programmatic interface that lets software communicate with a service.

Endpoint

The specific URL used to access an API operation.

HTTP GET

An HTTP method normally used to retrieve data.

JSON

A text-based structured data format commonly used in APIs.

Query parameter

A key-value input appended to a URL/request to specify what the API should return.

DataFrame

Pandas' two-dimensional tabular data structure.

ETL

Extract, Transform, Load: get data, clean/convert it, then store it.

Transaction

A group of database operations treated as one logical unit.

Commit

Permanently applies the current transaction's changes.

Rollback

Reverses uncommitted transaction changes.

Cursor

An object used to execute SQL through a database connection.

Primary key

A column or set of columns that uniquely identifies a row.

Unique constraint

A database rule preventing duplicate values for a specified column combination.

Index

A database structure that speeds up searches at the cost of storage/write overhead.

Parameterized SQL

SQL using placeholders with values supplied separately to the database driver.

SQL injection

An attack where malicious input changes the intended SQL command.

Idempotency

Repeated execution produces the same intended final state.

Logging

Structured recording of program events and errors.

Environment variable

Configuration stored outside the source code and supplied by the environment.

Power BI

A business-intelligence platform for interactive reports and dashboards.

DirectQuery

A Power BI mode where queries are sent to the source rather than importing all data into the model.

Incremental loading

Loading only new or changed data instead of reprocessing the entire history.

Retry/backoff

Retrying failed operations with increasing delays to handle temporary failures and rate limits.

Bulk insertion

Sending many rows to the database in batches instead of one SQL execution per row.

60-Second Project Answer

“My project is an end-to-end stock market analytics pipeline. I use Alpha Vantage as the source for daily stock data for RELIANCE.BSE, TCS.BSE, INFY.BSE and IBM. I built the ETL process in Python. The script loads configuration from environment variables, calls the API, validates the response, converts the JSON time series into a Pandas DataFrame, converts dates and numeric fields, removes invalid required records, and loads the data into MySQL. To prevent duplicate historical records, the load logic checks the symbol and date before insertion. Logging and exception handling make the pipeline easier to monitor and troubleshoot. MySQL provides persistent storage and the analytics layer, while Power BI provides market overview, stock performance and risk/trading dashboards. Windows Task Scheduler automates the ETL execution.”

Official References

Alpha Vantage — <https://www.alphavantage.co/documentation/>

Python logging — <https://docs.python.org/3/library/logging.html>

Pandas — <https://pandas.pydata.org/docs/>

Requests — <https://requests.readthedocs.io/>

MySQL Connector/Python — <https://dev.mysql.com/doc/connector-python/en/>

python-dotenv — <https://bbc2.github.io/python-dotenv/>